

AHDS - PYTHON-01

Python Programming & Engineering Reference

PART 3 - FUNCTIONS, PARAMETERS, SCOPE & FUNCTIONAL PROGRAMMING

Chapters 68-99 • English / Français

Engineering focus / Orientation ingénierie

Reusable function interfaces, argument design, scope, callbacks, generators and robust callable architecture for engineering software.
Interfaces de fonctions réutilisables, conception des arguments, portée, callbacks, générateurs et architecture robuste des objets appelables pour les logiciels d’ingénierie.

Editorial note / Note éditoriale
Verify version-specific APIs against the official documentation of the Python release used.
Vérifier les API dépendantes de la version dans la documentation officielle de la version Python utilisée.



© AHDS. Portfolio / educational material. Redistribution or reproduction without permission is not authorized.

Table des matières / Contents

Section A — Function Fundamentals / Fondamentaux des fonctions

68. Function Fundamentals / Fondamentaux des fonctions
69. Function Parameters & Arguments / Paramètres et arguments de fonction
70. Positional Arguments / Arguments positionnels
71. Keyword Arguments / Arguments nommés
72. Default Parameter Values / Valeurs par défaut des paramètres
73. Positional-Only Parameters — / / Paramètres positionnels uniquement — /
74. Keyword-Only Parameters — * / Paramètres nommés uniquement — *
75. Variable Positional Arguments — *args / Arguments positionnels variables — *args
76. Variable Keyword Arguments — **kwargs / Arguments nommés variables — **kwargs
77. Argument Packing & Unpacking / Regroupement et déballage des arguments

Section B — Results & API Design / Résultats et conception d'API

78. return & Function Results / return et résultats de fonction
79. Returning Multiple Values / Retour de plusieurs valeurs
80. Function Signatures & API Design / Signatures de fonction et conception d'API
81. Function Annotations / Annotations de fonction
82. Type Hints for Functions / Indications de type pour les fonctions

Section C — Scope & Namespaces / Portée et espaces de noms

83. Namespaces & Scope / Espaces de noms et portée
84. LEGB Name Resolution / Résolution des noms LEGB
85. global / global
86. nonlocal / nonlocal
87. Closures / Fermetures

Section D — Functions as Objects / Fonctions en tant qu'objets

88. First-Class Functions / Fonctions de première classe



© AHDS. Portfolio / educational material. Redistribution or reproduction without permission is not authorized.



89. Higher-Order Functions / Fonctions d’ordre supérieur

90. Lambda Expressions / Expressions lambda

91. Callbacks & Event-Oriented Functions / Callbacks et fonctions orientées événements

Section E — Recursion & Functional Programming / Récursivité et programmation fonctionnelle

92. Recursion / Récursivité

93. Functional Programming Fundamentals / Fondamentaux de la programmation fonctionnelle

94. Pure Functions, Side Effects & State / Fonctions pures, effets de bord et état

Section F — Iterators, Generators & Decorators / Itérateurs, générateurs et décorateurs

95. Custom Iterators / Itérateurs personnalisés

96. Generator Functions & yield / Fonctions génératrices et yield

97. yield from & Generator Delegation / yield from et délégation de générateur

98. Function Decorators / Décorateurs de fonction

Section G — Function Introspection & Robust Design / Introspection des fonctions et conception robuste

99. Function Introspection, Documentation & Robust Function Design / Introspection, documentation et conception robuste des fonctions



© AHDS. Portfolio / educational material. Redistribution or reproduction without permission is not authorized.



Section A — Function Fundamentals / Fondamentaux des fonctions

Chapter 68 — Function Fundamentals

Fondamentaux des fonctions

Engineering focus / Orientation ingénierie: def, calls, local execution and reusable behavior. / def, appels, exécution locale et comportement réutilisable.

ENGLISH	FRANÇAIS
What is the core purpose of function fundamentals? A function defines a reusable callable interface with explicit inputs, outputs and failure behavior.	**Quel est le rôle principal de « Fondamentaux des fonctions » ?** Une fonction définit une interface réutilisable avec des entrées, des sorties et un comportement en cas d'échec.

ENGLISH	FRANÇAIS
What design mistake should be avoided? Keep side effects separate from computation; validate ADC limits and reference voltage.	**Quelle erreur de conception faut-il éviter ?** Séparer E/S et calcul; vérifier les bornes ADC et la tension de référence.

ENGLISH	FRANÇAIS
How is this applied in engineering software? Apply this interface to instrument drivers and data pipelines; isolate hardware I/O, validate inputs and test failure paths.	**Comment l'appliquer dans un logiciel d'ingénierie ?** Appliquer cette interface aux pilotes et aux données; isoler les E/S, valider les entrées et tester les erreurs.

Engineering Notes	Notes d'ingénierie
Prefer explicit contracts, deterministic behavior and testable boundaries. Add type hints and documentation when they reduce ambiguity; do not use abstraction only for style.	Privilégier des contrats explicites, un comportement déterministe et des limites testables. Ajouter des indications de type et de la documentation lorsqu'elles réduisent l'ambiguïté; éviter l'abstraction purement stylistique.

Shared engineering example — code remains English only

```
def convert_adc(raw, vref=3.3):
    return raw * vref / 4095

voltage = convert_adc(2048)
```



© AHDS. Portfolio / educational material. Redistribution or reproduction without permission is not authorized.



Chapter 68 — Advanced Engineering Q&A / Questions avancées

ENGLISH	FRANÇAIS
How should an ADC conversion API handle invalid codes and reference voltage? Validate raw against the converter resolution and require a positive, finite VREF. Reject out-of-range inputs explicitly rather than silently clipping them; clipping can conceal acquisition faults. Define units and a testable error contract.	**Comment une API de conversion ADC doit-elle traiter des codes invalides et la tension de référence ?** Valider raw selon la résolution du convertisseur et exiger une tension VREF positive et finie. Rejeter explicitement les valeurs hors plage plutôt que les saturer silencieusement : la saturation peut masquer une défaillance. Définir les unités et les erreurs attendues.

ENGLISH	FRANÇAIS
How should an engineering function separate measurement I/O from numerical processing? Pass acquired samples into a pure conversion function instead of opening the device inside the calculation. This allows deterministic unit tests, simulated input, replay of logged measurements and independent hardware fault handling.	**Comment séparer les E/S de mesure du traitement numérique ?** Transmettre les échantillons à une fonction de conversion pure plutôt que d'accéder au matériel dans le calcul. On obtient des tests déterministes, des entrées simulées, la relecture des mesures et une gestion indépendante des défauts matériels.

ENGLISH	FRANÇAIS
What contract makes an ADC scaling function safe to reuse across devices? Specify ADC bit depth, conversion convention, reference source, signedness, units and handling of saturation. Do not hard-code a 12-bit denominator when the function is expected to support other converters.	**Quel contrat permet de réutiliser une fonction de mise à l'échelle ADC ?** Spécifier la résolution ADC, la convention de conversion, la référence, le signe, les unités et le traitement de la saturation. Ne pas coder en dur un diviseur 12 bits si d'autres convertisseurs doivent être acceptés.

Shared engineering example — code remains English only

```
import math

def adc_to_volts(raw: int, bits: int, vref: float) -> float:
    if not 1 <= bits <= 24 or not math.isfinite(vref) or vref <= 0:
        raise ValueError("Invalid ADC configuration")
    full_scale = (1 << bits) - 1
    if not 0 <= raw <= full_scale:
        raise ValueError("ADC code outside range")
    return raw * vref / full_scale
```



© AHDS. Portfolio / educational material. Redistribution or reproduction without permission is not authorized.



Chapter 69 — Function Parameters & Arguments

Paramètres et arguments de fonction

Engineering focus / Orientation ingénierie: parameters define the interface; arguments provide values at call time. / les paramètres définissent l'interface; les arguments fournissent les valeurs à l'appel.

ENGLISH	FRANÇAIS
What is the core purpose of function parameters & arguments? Parameters belong to a definition; arguments are supplied by a caller and bound at invocation.	**Quel est le rôle principal de « Paramètres et arguments de fonction » ?** Les paramètres figurent dans la définition; les arguments sont fournis et associés lors de l'appel.

ENGLISH	FRANÇAIS
What design mistake should be avoided? Reject invalid channel numbers and non-positive sample counts at the interface.	**Quelle erreur de conception faut-il éviter ?** Rejeter les canaux invalides et les nombres d'échantillons non positifs.

ENGLISH	FRANÇAIS
How is this applied in engineering software? Apply this interface to instrument drivers and data pipelines; isolate hardware I/O, validate inputs and test failure paths.	**Comment l'appliquer dans un logiciel d'ingénierie ?** Appliquer cette interface aux pilotes et aux données; isoler les E/S, valider les entrées et tester les erreurs.

Engineering Notes	Notes d'ingénierie
Prefer explicit contracts, deterministic behavior and testable boundaries. Add type hints and documentation when they reduce ambiguity; do not use abstraction only for style.	Privilégier des contrats explicites, un comportement déterministe et des limites testables. Ajouter des indications de type et de la documentation lorsqu'elles réduisent l'ambiguïté; éviter l'abstraction purement stylistique.

Shared engineering example — code remains English only

```
def measure(channel, samples):
    return [read_adc(channel) for _ in range(samples)]
```



© AHDS. Portfolio / educational material. Redistribution or reproduction without permission is not authorized.



Chapter 70 — Positional Arguments

Arguments positionnels

Engineering focus / Orientation ingénierie: binding by position and the importance of stable ordering. / liaison par position et importance d'un ordre stable.

ENGLISH	FRANÇAIS
What is the core purpose of positional arguments? Positional arguments bind by order, making argument sequence part of the callable contract.	**Quel est le rôle principal de « Arguments positionnels » ?** Les arguments positionnels sont associés selon leur ordre dans l'appel.

ENGLISH	FRANÇAIS
What design mistake should be avoided? Avoid swapping gain and offset; use keyword-only arguments when ambiguity is costly.	**Quelle erreur de conception faut-il éviter ?** Éviter d'inverser gain et décalage; imposer des noms si nécessaire.

ENGLISH	FRANÇAIS
How is this applied in engineering software? Apply this interface to instrument drivers and data pipelines; isolate hardware I/O, validate inputs and test failure paths.	**Comment l'appliquer dans un logiciel d'ingénierie ?** Appliquer cette interface aux pilotes et aux données; isoler les E/S, valider les entrées et tester les erreurs.

Engineering Notes	Notes d'ingénierie
Prefer explicit contracts, deterministic behavior and testable boundaries. Add type hints and documentation when they reduce ambiguity; do not use abstraction only for style.	Privilégier des contrats explicites, un comportement déterministe et des limites testables. Ajouter des indications de type et de la documentation lorsqu'elles réduisent l'ambiguïté; éviter l'abstraction purement stylistique.

Shared engineering example — code remains English only

```
def scale(value, gain, offset):
    return value * gain + offset
y = scale(2.4, 10.0, -1.0)
```



© AHDS. Portfolio / educational material. Redistribution or reproduction without permission is not authorized.



Chapter 71 — Keyword Arguments

Arguments nommés

Engineering focus / Orientation ingénierie: explicit call-site naming for readable engineering APIs. / nommage explicite à l'appel pour des API d'ingénierie lisibles.

ENGLISH	FRANÇAIS
What is the core purpose of keyword arguments? Keyword arguments bind by name and make configuration calls self-documenting.	**Quel est le rôle principal de « Arguments nommés » ?** Les arguments nommés sont associés par nom et clarifient la configuration.

ENGLISH	FRANÇAIS
What design mistake should be avoided? Validate port, baud rate and timeout rather than trusting descriptive names.	**Quelle erreur de conception faut-il éviter ?** Valider le port, le débit et le délai au lieu de se fier aux noms.

ENGLISH	FRANÇAIS
How is this applied in engineering software? Apply this interface to instrument drivers and data pipelines; isolate hardware I/O, validate inputs and test failure paths.	**Comment l'appliquer dans un logiciel d'ingénierie ?** Appliquer cette interface aux pilotes et aux données; isoler les E/S, valider les entrées et tester les erreurs.

Engineering Notes	Notes d'ingénierie
Prefer explicit contracts, deterministic behavior and testable boundaries. Add type hints and documentation when they reduce ambiguity; do not use abstraction only for style.	Privilégier des contrats explicites, un comportement déterministe et des limites testables. Ajouter des indications de type et de la documentation lorsqu'elles réduisent l'ambiguïté; éviter l'abstraction purement stylistique.

Shared engineering example — code remains English only

```
def configure(*, port, baud, timeout):
    return {"port": port, "baud": baud, "timeout": timeout}
settings = configure(port="COM4", baud=115200, timeout=0.5)
```



© AHDS. Portfolio / educational material. Redistribution or reproduction without permission is not authorized.



Chapter 72 — Default Parameter Values

Valeurs par défaut des paramètres

Engineering focus / Orientation ingénierie: safe defaults and the mutable-default trap. / valeurs par défaut sûres et piège des valeurs mutables.

ENGLISH	FRANÇAIS
What is the core purpose of default parameter values?	**Quel est le rôle principal de « Valeurs par défaut des paramètres » ?**
Default values are evaluated when the function is defined, not at each call.	Les valeurs par défaut sont évaluées à la définition, pas à chaque appel.

ENGLISH	FRANÇAIS
What design mistake should be avoided?	**Quelle erreur de conception faut-il éviter ?**
Never use a shared mutable list as an unintended default; distinguish None explicitly.	Éviter les listes mutables partagées; traiter explicitement None.

ENGLISH	FRANÇAIS
How is this applied in engineering software?	**Comment l'appliquer dans un logiciel d'ingénierie ?**
Apply this interface to instrument drivers and data pipelines; isolate hardware I/O, validate inputs and test failure paths.	Appliquer cette interface aux pilotes et aux données; isoler les E/S, valider les entrées et tester les erreurs.

Engineering Notes	Notes d'ingénierie
Prefer explicit contracts, deterministic behavior and testable boundaries. Add type hints and documentation when they reduce ambiguity; do not use abstraction only for style.	Privilégier des contrats explicites, un comportement déterministe et des limites testables. Ajouter des indications de type et de la documentation lorsqu'elles réduisent l'ambiguïté; éviter l'abstraction purement stylistique.

Shared engineering example — code remains English only

```
def log_event(msg, tags=None):
    tags = [] if tags is None else tags
    return {"msg": msg, "tags": tags}
```



© AHDS. Portfolio / educational material. Redistribution or reproduction without permission is not authorized.



Chapter 73 — Positional-Only Parameters — /

Paramètres positionnels uniquement — /

Engineering focus / Orientation ingénierie: using / to protect APIs from unnecessary parameter-name coupling. / utilisation de / pour éviter un couplage inutile aux noms de paramètres.

ENGLISH	FRANÇAIS
What is the core purpose of positional-only parameters — /? The / separator makes preceding parameters positional-only, decoupling an API from their names.	**Quel est le rôle principal de « Paramètres positionnels uniquement — / » ?** Le séparateur / impose les arguments positionnels avant lui et stabilise l'API.

ENGLISH	FRANÇAIS
What design mistake should be avoided? Document argument order and reject reversed interval bounds.	**Quelle erreur de conception faut-il éviter ?** Documenter l'ordre et rejeter des bornes inversées.

ENGLISH	FRANÇAIS
How is this applied in engineering software? Apply this interface to instrument drivers and data pipelines; isolate hardware I/O, validate inputs and test failure paths.	**Comment l'appliquer dans un logiciel d'ingénierie ?** Appliquer cette interface aux pilotes et aux données; isoler les E/S, valider les entrées et tester les erreurs.

Engineering Notes	Notes d'ingénierie
Prefer explicit contracts, deterministic behavior and testable boundaries. Add type hints and documentation when they reduce ambiguity; do not use abstraction only for style.	Privilégier des contrats explicites, un comportement déterministe et des limites testables. Ajouter des indications de type et de la documentation lorsqu'elles réduisent l'ambiguïté; éviter l'abstraction purement stylistique.

Shared engineering example — code remains English only

```
def clamp(value, minimum, maximum, /):
    return max(minimum, min(value, maximum))
```



© AHDS. Portfolio / educational material. Redistribution or reproduction without permission is not authorized.



Chapter 74 — Keyword-Only Parameters — *

Paramètres nommés uniquement — *

Engineering focus / Orientation ingénierie: forcing safety-critical options to be named. / obligation de nommer les options critiques pour la sécurité.

ENGLISH	FRANÇAIS
What is the core purpose of keyword-only parameters — *? The * separator forces following parameters to be passed by name.	**Quel est le rôle principal de « Paramètres nommés uniquement — * » ?** Le séparateur * impose des arguments nommés pour les paramètres suivants.

ENGLISH	FRANÇAIS
What design mistake should be avoided? Use named timeouts and limits to avoid accidental configuration mistakes.	**Quelle erreur de conception faut-il éviter ?** Nommer délais et limites pour éviter les erreurs de configuration.

ENGLISH	FRANÇAIS
How is this applied in engineering software? Apply this interface to instrument drivers and data pipelines; isolate hardware I/O, validate inputs and test failure paths.	**Comment l'appliquer dans un logiciel d'ingénierie ?** Appliquer cette interface aux pilotes et aux données; isoler les E/S, valider les entrées et tester les erreurs.

Engineering Notes	Notes d'ingénierie
Prefer explicit contracts, deterministic behavior and testable boundaries. Add type hints and documentation when they reduce ambiguity; do not use abstraction only for style.	Privilégier des contrats explicites, un comportement déterministe et des limites testables. Ajouter des indications de type et de la documentation lorsqu'elles réduisent l'ambiguïté; éviter l'abstraction purement stylistique.

Shared engineering example — code remains English only

```
def connect(port, *, baud=115200, timeout=1.0):
    if timeout <= 0: raise ValueError("timeout must be positive")
    return {"port": port, "baud": baud, "timeout": timeout}
```



© AHDS. Portfolio / educational material. Redistribution or reproduction without permission is not authorized.



Chapter 75 — Variable Positional Arguments — *args

Arguments positionnels variables — *args

Engineering focus / Orientation ingénierie: collecting a variable number of positional inputs without hiding semantics. / collecte d'un nombre variable d'entrées positionnelles sans masquer la sémantique.

ENGLISH	FRANÇAIS
What is the core purpose of variable positional arguments — *args? *args gathers positional inputs into a tuple for variable-arity functions.	**Quel est le rôle principal de « Arguments positionnels variables — *args » ?** *args regroupe les entrées positionnelles dans un tuple.

ENGLISH	FRANÇAIS
What design mistake should be avoided? Define behavior for zero values and reject incompatible sample types.	**Quelle erreur de conception faut-il éviter ?** Préciser le cas sans valeur et rejeter les types incompatibles.

ENGLISH	FRANÇAIS
How is this applied in engineering software? Apply this interface to instrument drivers and data pipelines; isolate hardware I/O, validate inputs and test failure paths.	**Comment l'appliquer dans un logiciel d'ingénierie ?** Appliquer cette interface aux pilotes et aux données; isoler les E/S, valider les entrées et tester les erreurs.

Engineering Notes	Notes d'ingénierie
Prefer explicit contracts, deterministic behavior and testable boundaries. Add type hints and documentation when they reduce ambiguity; do not use abstraction only for style.	Privilégier des contrats explicites, un comportement déterministe et des limites testables. Ajouter des indications de type et de la documentation lorsqu'elles réduisent l'ambiguïté; éviter l'abstraction purement stylistique.

Shared engineering example — code remains English only

```
def average(*values):
    if not values: raise ValueError("no samples")
    return sum(values) / len(values)
```



© AHDS. Portfolio / educational material. Redistribution or reproduction without permission is not authorized.



Chapter 76 — Variable Keyword Arguments — **kwargs

Arguments nommés variables — **kwargs

Engineering focus / Orientation ingénierie: extensible option forwarding with validation. / transmission extensible d'options avec validation.

ENGLISH	FRANÇAIS
What is the core purpose of variable keyword arguments — **kwargs? **kwargs collects otherwise unbound keyword arguments in a dictionary.	**Quel est le rôle principal de « Arguments nommés variables — **kwargs » ?** **kwargs collecte les arguments nommés non liés dans un dictionnaire.

ENGLISH	FRANÇAIS
What design mistake should be avoided? Whitelist supported options; fail on unknown keys rather than silently ignoring them.	**Quelle erreur de conception faut-il éviter ?** Autoriser les options connues; refuser les clés inconnues.

ENGLISH	FRANÇAIS
How is this applied in engineering software? Apply this interface to instrument drivers and data pipelines; isolate hardware I/O, validate inputs and test failure paths.	**Comment l'appliquer dans un logiciel d'ingénierie ?** Appliquer cette interface aux pilotes et aux données; isoler les E/S, valider les entrées et tester les erreurs.

Engineering Notes	Notes d'ingénierie
Prefer explicit contracts, deterministic behavior and testable boundaries. Add type hints and documentation when they reduce ambiguity; do not use abstraction only for style.	Privilégier des contrats explicites, un comportement déterministe et des limites testables. Ajouter des indications de type et de la documentation lorsqu'elles réduisent l'ambiguïté; éviter l'abstraction purement stylistique.

Shared engineering example — code remains English only

```
def configure(**options):
    allowed = {"baud", "timeout"}
    unknown = options.keys() - allowed
    if unknown: raise TypeError(f"Unknown options: {unknown}")
    return options
```



© AHDS. Portfolio / educational material. Redistribution or reproduction without permission is not authorized.



Chapter 77 — Argument Packing & Unpacking

Regroupement et déballage des arguments

Engineering focus / Orientation ingénierie: * and ** at call boundaries for structured data. / * et ** aux limites d'appel pour des données structurées.

ENGLISH	FRANÇAIS
What is the core purpose of argument packing & unpacking? At a call site, * expands iterables and ** expands mappings into arguments.	**Quel est le rôle principal de « Regroupement et déballage des arguments » ?** À l'appel, * développe un itérable et ** développe un dictionnaire.

ENGLISH	FRANÇAIS
What design mistake should be avoided? Check signatures and prevent duplicate arguments or missing required fields.	**Quelle erreur de conception faut-il éviter ?** Vérifier les signatures; éviter les doublons et champs manquants.

ENGLISH	FRANÇAIS
How is this applied in engineering software? Apply this interface to instrument drivers and data pipelines; isolate hardware I/O, validate inputs and test failure paths.	**Comment l'appliquer dans un logiciel d'ingénierie ?** Appliquer cette interface aux pilotes et aux données; isoler les E/S, valider les entrées et tester les erreurs.

Engineering Notes	Notes d'ingénierie
Prefer explicit contracts, deterministic behavior and testable boundaries. Add type hints and documentation when they reduce ambiguity; do not use abstraction only for style.	Privilégier des contrats explicites, un comportement déterministe et des limites testables. Ajouter des indications de type et de la documentation lorsqu'elles réduisent l'ambiguïté; éviter l'abstraction purement stylistique.

Shared engineering example — code remains English only

```
args = (2.4, 10.0, -1.0)
y = scale(*args)
opts = {"baud": 115200, "timeout": 0.5}
connect("COM4", **opts)
```



© AHDS. Portfolio / educational material. Redistribution or reproduction without permission is not authorized.



Section B — Results & API Design / Résultats et conception d'API

Chapter 78 — return & Function Results

return et résultats de fonction

Engineering focus / Orientation ingénierie: explicit result contracts and early returns. / contrats de résultat explicites et retours anticipés.

ENGLISH	FRANÇAIS
What is the core purpose of return & function results? return ends a function and supplies its result; reaching the end returns None.	**Quel est le rôle principal de « return et résultats de fonction » ?** return termine la fonction et fournit son résultat; sans return, on obtient None.

ENGLISH	FRANÇAIS
What design mistake should be avoided? Use explicit status contracts; do not confuse invalid data with a valid zero.	**Quelle erreur de conception faut-il éviter ?** Définir un contrat de résultat et distinguer erreur et valeur zéro.

ENGLISH	FRANÇAIS
How is this applied in engineering software? Apply this interface to instrument drivers and data pipelines; isolate hardware I/O, validate inputs and test failure paths.	**Comment l'appliquer dans un logiciel d'ingénierie ?** Appliquer cette interface aux pilotes et aux données; isoler les E/S, valider les entrées et tester les erreurs.

Engineering Notes	Notes d'ingénierie
Prefer explicit contracts, deterministic behavior and testable boundaries. Add type hints and documentation when they reduce ambiguity; do not use abstraction only for style.	Privilégier des contrats explicites, un comportement déterministe et des limites testables. Ajouter des indications de type et de la documentation lorsqu'elles réduisent l'ambiguïté; éviter l'abstraction purement stylistique.

Shared engineering example — code remains English only

```
def validate(v):
    if not 0 <= v <= 5:
        return False
    return True
```



© AHDS. Portfolio / educational material. Redistribution or reproduction without permission is not authorized.



Chapter 79 — Returning Multiple Values

Retour de plusieurs valeurs

Engineering focus / Orientation ingénierie: tuple-based multi-result APIs and unpacking. / API à résultats multiples basées sur les tuples et déballage.

ENGLISH	FRANÇAIS
What is the core purpose of returning multiple values? Returning comma-separated values constructs a tuple that callers can unpack.	**Quel est le rôle principal de « Retour de plusieurs valeurs » ?** Retourner plusieurs valeurs crée un tuple pouvant être décomposé.

ENGLISH	FRANÇAIS
What design mistake should be avoided? Reject empty measurements before min, max and mean calculations.	**Quelle erreur de conception faut-il éviter ?** Rejeter les mesures vides avant de calculer min, max et moyenne.

ENGLISH	FRANÇAIS
How is this applied in engineering software? Apply this interface to instrument drivers and data pipelines; isolate hardware I/O, validate inputs and test failure paths.	**Comment l'appliquer dans un logiciel d'ingénierie ?** Appliquer cette interface aux pilotes et aux données; isoler les E/S, valider les entrées et tester les erreurs.

Engineering Notes	Notes d'ingénierie
Prefer explicit contracts, deterministic behavior and testable boundaries. Add type hints and documentation when they reduce ambiguity; do not use abstraction only for style.	Privilégier des contrats explicites, un comportement déterministe et des limites testables. Ajouter des indications de type et de la documentation lorsqu'elles réduisent l'ambiguïté; éviter l'abstraction purement stylistique.

Shared engineering example — code remains English only

```
def stats(samples):
    if not samples: raise ValueError("empty measurements")
    return min(samples), max(samples), sum(samples) / len(samples)
lo, hi, avg = stats([1.1, 1.3, 1.2])
```



© AHDS. Portfolio / educational material. Redistribution or reproduction without permission is not authorized.



Chapter 80 — Function Signatures & API Design

Signatures de fonction et conception d’API

Engineering focus / Orientation ingénierie: clear signatures as executable interface documentation. / signatures claires comme documentation exécutable de l’interface.

ENGLISH	FRANÇAIS
What is the core purpose of function signatures & api design?	**Quel est le rôle principal de « Signatures de fonction et conception d’API » ?**
A well-designed signature separates required data from optional configuration.	Une bonne signature sépare les données obligatoires des options.

ENGLISH	FRANÇAIS
What design mistake should be avoided?	**Quelle erreur de conception faut-il éviter ?**
Specify units, allowed ranges, timeout semantics and error conditions.	Préciser unités, plages, délais et conditions d’erreur.

ENGLISH	FRANÇAIS
How is this applied in engineering software?	**Comment l’appliquer dans un logiciel d’ingénierie ?**
Apply this interface to instrument drivers and data pipelines; isolate hardware I/O, validate inputs and test failure paths.	Appliquer cette interface aux pilotes et aux données; isoler les E/S, valider les entrées et tester les erreurs.

Engineering Notes	Notes d’ingénierie
Prefer explicit contracts, deterministic behavior and testable boundaries. Add type hints and documentation when they reduce ambiguity; do not use abstraction only for style.	Privilégier des contrats explicites, un comportement déterministe et des limites testables. Ajouter des indications de type et de la documentation lorsqu’elles réduisent l’ambiguïté; éviter l’abstraction purement stylistique.

Shared engineering example — code remains English only

```
def acquire(channel: int, *, samples: int = 100, timeout: float = 1.0):
    if channel < 0 or samples < 1 or timeout <= 0:
        raise ValueError("invalid acquisition settings")
    return {"channel": channel, "samples": samples}
```



© AHDS. Portfolio / educational material. Redistribution or reproduction without permission is not authorized.



Chapter 81 — Function Annotations

Annotations de fonction

Engineering focus / Orientation ingénierie: metadata on parameters and returns without runtime enforcement by Python itself. / métadonnées sur paramètres et retours sans validation automatique par Python.

ENGLISH	FRANÇAIS
What is the core purpose of function annotations? Annotations attach metadata to arguments and results; Python does not enforce types automatically.	**Quel est le rôle principal de « Annotations de fonction » ?** Les annotations décrivent arguments et retours; Python ne vérifie pas les types seul.

ENGLISH	FRANÇAIS
What design mistake should be avoided? Do not mistake annotations for runtime validation of sensor values.	**Quelle erreur de conception faut-il éviter ?** Ne pas confondre annotations et validation des mesures.

ENGLISH	FRANÇAIS
How is this applied in engineering software? Apply this interface to instrument drivers and data pipelines; isolate hardware I/O, validate inputs and test failure paths.	**Comment l'appliquer dans un logiciel d'ingénierie ?** Appliquer cette interface aux pilotes et aux données; isoler les E/S, valider les entrées et tester les erreurs.

Engineering Notes	Notes d'ingénierie
Prefer explicit contracts, deterministic behavior and testable boundaries. Add type hints and documentation when they reduce ambiguity; do not use abstraction only for style.	Privilégier des contrats explicites, un comportement déterministe et des limites testables. Ajouter des indications de type et de la documentation lorsqu'elles réduisent l'ambiguïté; éviter l'abstraction purement stylistique.

Shared engineering example — code remains English only

```
def volts(raw: int, vref: float) -> float:
    return raw * vref / 4095
```



© AHDS. Portfolio / educational material. Redistribution or reproduction without permission is not authorized.



Chapter 82 — Type Hints for Functions

Indications de type pour les fonctions

Engineering focus / Orientation ingénierie: static analysis, IDE support and maintainable interfaces. / analyse statique, aide IDE et interfaces maintenables.

ENGLISH	FRANÇAIS
What is the core purpose of type hints for functions?	**Quel est le rôle principal de « Indications de type pour les fonctions » ?**
Type hints describe interface expectations for static analyzers and IDEs.	Les indications de type décrivent les contrats pour les analyseurs et IDE.

ENGLISH	FRANÇAIS
What design mistake should be avoided?	**Quelle erreur de conception faut-il éviter ?**
Handle empty input; check runtime validity at external data boundaries.	Traiter les entrées vides et valider les données externes.

ENGLISH	FRANÇAIS
How is this applied in engineering software?	**Comment l'appliquer dans un logiciel d'ingénierie ?**
Apply this interface to instrument drivers and data pipelines; isolate hardware I/O, validate inputs and test failure paths.	Appliquer cette interface aux pilotes et aux données; isoler les E/S, valider les entrées et tester les erreurs.

Engineering Notes	Notes d'ingénierie
Prefer explicit contracts, deterministic behavior and testable boundaries. Add type hints and documentation when they reduce ambiguity; do not use abstraction only for style.	Privilégier des contrats explicites, un comportement déterministe et des limites testables. Ajouter des indications de type et de la documentation lorsqu'elles réduisent l'ambiguïté; éviter l'abstraction purement stylistique.

Shared engineering example — code remains English only

```
from collections.abc import Sequence
from math import sqrt
def rms(values: Sequence[float]) -> float:
    if not values: raise ValueError("empty input")
    return sqrt(sum(x*x for x in values) / len(values))
```



© AHDS. Portfolio / educational material. Redistribution or reproduction without permission is not authorized.



Section C — Scope & Namespaces / Portée et espaces de noms

Chapter 83 — Namespaces & Scope

Espaces de noms et portée

Engineering focus / Orientation ingénierie: where names live and how function calls create local namespaces. / emplacement des noms et création d'espaces locaux par les appels.

ENGLISH	FRANÇAIS
What is the core purpose of namespaces & scope? Namespaces map identifiers to objects; scope determines where names can be resolved.	**Quel est le rôle principal de « Espaces de noms et portée » ?** Les espaces de noms associent des noms aux objets; la portée détermine leur visibilité.

ENGLISH	FRANÇAIS
What design mistake should be avoided? Avoid hidden coupling to mutable module configuration in drivers.	**Quelle erreur de conception faut-il éviter ?** Éviter le couplage caché avec la configuration globale mutable.

ENGLISH	FRANÇAIS
How is this applied in engineering software? Apply this interface to instrument drivers and data pipelines; isolate hardware I/O, validate inputs and test failure paths.	**Comment l'appliquer dans un logiciel d'ingénierie ?** Appliquer cette interface aux pilotes et aux données; isoler les E/S, valider les entrées et tester les erreurs.

Engineering Notes	Notes d'ingénierie
Prefer explicit contracts, deterministic behavior and testable boundaries. Add type hints and documentation when they reduce ambiguity; do not use abstraction only for style.	Privilégier des contrats explicites, un comportement déterministe et des limites testables. Ajouter des indications de type et de la documentation lorsqu'elles réduisent l'ambiguïté; éviter l'abstraction purement stylistique.

Shared engineering example — code remains English only

```
limit = 5.0
def check(value):
    status = value <= limit
    return status
```



© AHDS. Portfolio / educational material. Redistribution or reproduction without permission is not authorized.



Chapter 83 — Advanced Engineering Q&A / Questions avancées

ENGLISH	FRANÇAIS
Why can mutable module-level state make concurrent acquisition unreliable? Two acquisition tasks can update shared counters or buffers in an unexpected order. Prefer state owned by a session object, clear synchronization boundaries and explicit ownership; test concurrent callbacks and recovery paths.	**Pourquoi un état global mutable fragilise-t-il une acquisition concurrente ?** Deux tâches peuvent modifier des compteurs ou tampons partagés dans un ordre inattendu. Préférer un état propre à chaque session, une synchronisation explicite et une propriété claire; tester les callbacks concurrents et la reprise.

ENGLISH	FRANÇAIS
What distinguishes a closure from a mutable global configuration? A closure captures names from an enclosing lexical scope. Independent factory calls can therefore hold independent state without exposing a module-global variable. Captured mutable objects still need careful sharing and synchronization.	**Quelle différence entre une fermeture et une configuration globale mutable ?** Une fermeture capture des noms de la portée lexicale englobante. Chaque appel de la fabrique peut donc conserver un état indépendant, sans variable globale. Les objets mutables capturés exigent néanmoins de gérer le partage et la synchronisation.

ENGLISH	FRANÇAIS
How can dependency injection improve testability at the scope boundary? Pass a read function or device adapter as an argument. Production uses a hardware driver; tests substitute a deterministic fake. No implicit global device handle is needed, and failures remain observable at the interface.	**Comment l'injection de dépendances améliore-t-elle les tests liés à la portée ?** Passer une fonction de lecture ou un adaptateur comme argument. La production utilise le pilote matériel; les tests utilisent un substitut déterministe. Aucun accès global implicite au périphérique n'est nécessaire et les erreurs restent visibles.

Shared engineering example — code remains English only

```
def make_sampler(read_channel):
    count = 0
    def sample(channel):
        nonlocal count
        result = read_channel(channel)
        count += 1
        return count, result
    return sample
```



© AHDS. Portfolio / educational material. Redistribution or reproduction without permission is not authorized.



Chapter 84 — LEGB Name Resolution

Résolution des noms LEGB

Engineering focus / Orientation ingénierie: Local, Enclosing, Global and Builtins lookup order. / ordre de recherche Local, Englobant, Global et Builtins.

ENGLISH	FRANÇAIS
What is the core purpose of legb name resolution? Name lookup generally follows Local, Enclosing, Global, then Builtins (LEGB).	**Quel est le rôle principal de « Résolution des noms LEGB » ?** La recherche suit généralement Local, Englobant, Global, puis Builtins.

ENGLISH	FRANÇAIS
What design mistake should be avoided? Avoid shadowing a calibrated gain or built-in function unintentionally.	**Quelle erreur de conception faut-il éviter ?** Éviter de masquer accidentellement un gain ou une fonction intégrée.

ENGLISH	FRANÇAIS
How is this applied in engineering software? Apply this interface to instrument drivers and data pipelines; isolate hardware I/O, validate inputs and test failure paths.	**Comment l'appliquer dans un logiciel d'ingénierie ?** Appliquer cette interface aux pilotes et aux données; isoler les E/S, valider les entrées et tester les erreurs.

Engineering Notes	Notes d'ingénierie
Prefer explicit contracts, deterministic behavior and testable boundaries. Add type hints and documentation when they reduce ambiguity; do not use abstraction only for style.	Privilégier des contrats explicites, un comportement déterministe et des limites testables. Ajouter des indications de type et de la documentation lorsqu'elles réduisent l'ambiguïté; éviter l'abstraction purement stylistique.

Shared engineering example — code remains English only

```
gain = 10
def outer():
    offset = 1
    def inner(x):
        return x * gain + offset
    return inner
```



© AHDS. Portfolio / educational material. Redistribution or reproduction without permission is not authorized.



Chapter 84 — Advanced Engineering Q&A / Questions avancées

ENGLISH	FRANÇAIS
Why do callbacks created inside a loop often use the last channel? Closures resolve free variables when invoked, not when the lambda is created. Loop-created callbacks may all refer to the same final channel variable. Bind each channel through a factory or a default parameter.	**Pourquoi des callbacks créés dans une boucle utilisent-ils souvent le dernier canal ?** Les variables libres sont résolues lors de l'appel, pas lors de la création du lambda. Tous les callbacks peuvent donc référencer la dernière valeur du canal. Fixer chaque canal par une fabrique ou un paramètre par défaut.

ENGLISH	FRANÇAIS
How does LEGB lookup interact with local assignment? Any assignment to a name in a function generally makes it local in that function. Reading it before assignment can raise UnboundLocalError even if a global of the same name exists. Use explicit parameters instead of implicit fallback.	**Comment LEGB interagit-il avec une affectation locale ?** Une affectation rend normalement le nom local à la fonction. Une lecture avant l'affectation peut lever UnboundLocalError même si un nom global existe. Préférer les paramètres explicites à une recherche implicite.

ENGLISH	FRANÇAIS
When should an engineer prefer a callable object to a closure? When state needs inspection, serialization, reset or explicit lifecycle management, a small callable class may provide a clearer contract. Closures remain useful for lightweight adapters with limited encapsulated state.	**Quand préférer un objet appelable à une fermeture ?** Si l'état doit être inspecté, sérialisé, réinitialisé ou géré dans un cycle de vie explicite, une petite classe appelable est souvent plus claire. Les fermetures conviennent aux adaptateurs simples avec peu d'état.

Shared engineering example — code remains English only

```
# Each callback must retain its own channel.
callbacks = [lambda ch=channel: read_adc(ch)
        for channel in range(16)]
# Contrasting bug: lambda: read_adc(channel) binds late.
```



© AHDS. Portfolio / educational material. Redistribution or reproduction without permission is not authorized.



Chapter 85 — global

global

Engineering focus / Orientation ingénierie: explicit rebinding of a module-level name and why shared mutable state should be minimized. / réaffectation explicite d'un nom de module et réduction de l'état global partagé.

ENGLISH	FRANÇAIS
What is the core purpose of global? global declares that rebinding a name refers to the module namespace.	**Quel est le rôle principal de « global » ?** global déclare qu'une réaffectation vise l'espace de noms du module.

ENGLISH	FRANÇAIS
What design mistake should be avoided? Shared global counters need synchronization under concurrency.	**Quelle erreur de conception faut-il éviter ?** Les compteurs globaux partagés exigent une synchronisation.

ENGLISH	FRANÇAIS
How is this applied in engineering software? Apply this interface to instrument drivers and data pipelines; isolate hardware I/O, validate inputs and test failure paths.	**Comment l'appliquer dans un logiciel d'ingénierie ?** Appliquer cette interface aux pilotes et aux données; isoler les E/S, valider les entrées et tester les erreurs.

Engineering Notes	Notes d'ingénierie
Prefer explicit contracts, deterministic behavior and testable boundaries. Add type hints and documentation when they reduce ambiguity; do not use abstraction only for style.	Privilégier des contrats explicites, un comportement déterministe et des limites testables. Ajouter des indications de type et de la documentation lorsqu'elles réduisent l'ambiguïté; éviter l'abstraction purement stylistique.

Shared engineering example — code remains English only

```
fault_count = 0
def record_fault():
    global fault_count
    fault_count += 1
```



© AHDS. Portfolio / educational material. Redistribution or reproduction without permission is not authorized.



Chapter 86 — nonlocal

nonlocal

Engineering focus / Orientation ingénierie: rebinding a name in an enclosing function scope. / réaffectation d'un nom dans la portée englobante.

ENGLISH	FRANÇAIS
What is the core purpose of nonlocal? nonlocal rebinds an existing variable from an enclosing function scope.	**Quel est le rôle principal de « nonlocal » ?** nonlocal réaffecte une variable de la fonction englobante.

ENGLISH	FRANÇAIS
What design mistake should be avoided? Remember closure state may persist across calls and require synchronization.	**Quelle erreur de conception faut-il éviter ?** L'état capturé persiste et peut nécessiter une synchronisation.

ENGLISH	FRANÇAIS
How is this applied in engineering software? Apply this interface to instrument drivers and data pipelines; isolate hardware I/O, validate inputs and test failure paths.	**Comment l'appliquer dans un logiciel d'ingénierie ?** Appliquer cette interface aux pilotes et aux données; isoler les E/S, valider les entrées et tester les erreurs.

Engineering Notes	Notes d'ingénierie
Prefer explicit contracts, deterministic behavior and testable boundaries. Add type hints and documentation when they reduce ambiguity; do not use abstraction only for style.	Privilégier des contrats explicites, un comportement déterministe et des limites testables. Ajouter des indications de type et de la documentation lorsqu'elles réduisent l'ambiguïté; éviter l'abstraction purement stylistique.

Shared engineering example — code remains English only

```
def counter():
    n = 0
    def inc():
        nonlocal n
        n += 1
        return n
    return inc
```



© AHDS. Portfolio / educational material. Redistribution or reproduction without permission is not authorized.



Chapter 87 — Closures

Fermetures

Engineering focus / Orientation ingénierie: functions retaining access to enclosing state after the outer call returns. / fonctions conservant l'accès à l'état englobant après le retour de la fonction externe.

ENGLISH	FRANÇAIS
What is the core purpose of closures? A closure retains access to non-global bindings from its defining scope.	**Quel est le rôle principal de « Fermetures » ?** Une fermeture conserve l'accès aux variables de sa portée englobante.

ENGLISH	FRANÇAIS
What design mistake should be avoided? Avoid accidental mutable shared state and late-binding surprises.	**Quelle erreur de conception faut-il éviter ?** Éviter l'état mutable partagé et les liaisons tardives inattendues.

ENGLISH	FRANÇAIS
How is this applied in engineering software? Apply this interface to instrument drivers and data pipelines; isolate hardware I/O, validate inputs and test failure paths.	**Comment l'appliquer dans un logiciel d'ingénierie ?** Appliquer cette interface aux pilotes et aux données; isoler les E/S, valider les entrées et tester les erreurs.

Engineering Notes	Notes d'ingénierie
Prefer explicit contracts, deterministic behavior and testable boundaries. Add type hints and documentation when they reduce ambiguity; do not use abstraction only for style.	Privilégier des contrats explicites, un comportement déterministe et des limites testables. Ajouter des indications de type et de la documentation lorsqu'elles réduisent l'ambiguïté; éviter l'abstraction purement stylistique.

Shared engineering example — code remains English only

```
def make_scaler(gain):
    def scale(x):
        return x * gain
    return scale
scale_10 = make_scaler(10)
```



© AHDS. Portfolio / educational material. Redistribution or reproduction without permission is not authorized.



Section D — Functions as Objects / Fonctions en tant qu'objets

Chapter 88 — First-Class Functions

Fonctions de première classe

Engineering focus / Orientation ingénierie: storing, passing and returning functions like other objects. / stockage, transmission et retour de fonctions comme d'autres objets.

ENGLISH	FRANÇAIS
What is the core purpose of first-class functions? Functions are first-class objects: they can be stored, passed and returned.	**Quel est le rôle principal de « Fonctions de première classe » ?** Les fonctions sont des objets de première classe, stockables et transmissibles.

ENGLISH	FRANÇAIS
What design mistake should be avoided? Validate dispatch keys; explicitly define handlers before invoking them.	**Quelle erreur de conception faut-il éviter ?** Valider les clés de routage et définir les gestionnaires avant usage.

ENGLISH	FRANÇAIS
How is this applied in engineering software? Apply this interface to instrument drivers and data pipelines; isolate hardware I/O, validate inputs and test failure paths.	**Comment l'appliquer dans un logiciel d'ingénierie ?** Appliquer cette interface aux pilotes et aux données; isoler les E/S, valider les entrées et tester les erreurs.

Engineering Notes	Notes d'ingénierie
Prefer explicit contracts, deterministic behavior and testable boundaries. Add type hints and documentation when they reduce ambiguity; do not use abstraction only for style.	Privilégier des contrats explicites, un comportement déterministe et des limites testables. Ajouter des indications de type et de la documentation lorsqu'elles réduisent l'ambiguïté; éviter l'abstraction purement stylistique.

Shared engineering example — code remains English only

```
def read_temp(): return 23.5
def read_vbat(): return 12.2
handlers = {"TEMP": read_temp, "VBAT": read_vbat}
value = handlers["TEMP"]()
```



© AHDS. Portfolio / educational material. Redistribution or reproduction without permission is not authorized.



Chapter 89 — Higher-Order Functions

Fonctions d’ordre supérieur

Engineering focus / Orientation ingénierie: functions that accept or return callables. / fonctions qui acceptent ou retournent des objets appelables.

ENGLISH	FRANÇAIS
What is the core purpose of higher-order functions? Higher-order functions accept or return callables, enabling reusable control flow.	**Quel est le rôle principal de « Fonctions d’ordre supérieur » ?** Les fonctions d’ordre supérieur reçoivent ou renvoient des fonctions.

ENGLISH	FRANÇAIS
What design mistake should be avoided? Specify retryable errors, bounded attempts and backoff policy.	**Quelle erreur de conception faut-il éviter ?** Définir erreurs réessayables, tentatives limitées et temporisation.

ENGLISH	FRANÇAIS
How is this applied in engineering software? Apply this interface to instrument drivers and data pipelines; isolate hardware I/O, validate inputs and test failure paths.	**Comment l’appliquer dans un logiciel d’ingénierie ?** Appliquer cette interface aux pilotes et aux données; isoler les E/S, valider les entrées et tester les erreurs.

Engineering Notes	Notes d’ingénierie
Prefer explicit contracts, deterministic behavior and testable boundaries. Add type hints and documentation when they reduce ambiguity; do not use abstraction only for style.	Privilégier des contrats explicites, un comportement déterministe et des limites testables. Ajouter des indications de type et de la documentation lorsqu’elles réduisent l’ambiguïté; éviter l’abstraction purement stylistique.

Shared engineering example — code remains English only

```
def retry(operation, attempts=3):
    for _ in range(attempts):
        if operation(): return True
    return False
```



© AHDS. Portfolio / educational material. Redistribution or reproduction without permission is not authorized.



Chapter 90 — Lambda Expressions

Expressions lambda

Engineering focus / Orientation ingénierie: small anonymous callables and when named functions are clearer. / petits objets appelables anonymes et cas où une fonction nommée est plus claire.

ENGLISH	FRANÇAIS
What is the core purpose of lambda expressions? lambda creates a small anonymous function containing a single expression.	**Quel est le rôle principal de « Expressions lambda » ?** lambda crée une fonction anonyme limitée à une expression.

ENGLISH	FRANÇAIS
What design mistake should be avoided? Prefer named functions for complex parsing or critical behavior.	**Quelle erreur de conception faut-il éviter ?** Préférer une fonction nommée pour les traitements critiques.

ENGLISH	FRANÇAIS
How is this applied in engineering software? Apply this interface to instrument drivers and data pipelines; isolate hardware I/O, validate inputs and test failure paths.	**Comment l'appliquer dans un logiciel d'ingénierie ?** Appliquer cette interface aux pilotes et aux données; isoler les E/S, valider les entrées et tester les erreurs.

Engineering Notes	Notes d'ingénierie
Prefer explicit contracts, deterministic behavior and testable boundaries. Add type hints and documentation when they reduce ambiguity; do not use abstraction only for style.	Privilégier des contrats explicites, un comportement déterministe et des limites testables. Ajouter des indications de type et de la documentation lorsqu'elles réduisent l'ambiguïté; éviter l'abstraction purement stylistique.

Shared engineering example — code remains English only

```
records.sort(key=lambda r: r["timestamp"])
```



© AHDS. Portfolio / educational material. Redistribution or reproduction without permission is not authorized.



Chapter 91 — Callbacks & Event-Oriented Functions

Callbacks et fonctions orientées événements

Engineering focus / Orientation ingénierie: decoupling event sources from handlers. / découplage des sources d'événements et des gestionnaires.

ENGLISH	FRANÇAIS
What is the core purpose of callbacks & event-oriented functions? Callbacks separate event producers from event handlers.	**Quel est le rôle principal de « Callbacks et fonctions orientées événements » ?** Les callbacks séparent la source des événements de leur traitement.

ENGLISH	FRANÇAIS
What design mistake should be avoided? Bound execution time and decide how exceptions in callbacks are handled.	**Quelle erreur de conception faut-il éviter ?** Limiter le temps d'exécution et traiter les exceptions des callbacks.

ENGLISH	FRANÇAIS
How is this applied in engineering software? Apply this interface to instrument drivers and data pipelines; isolate hardware I/O, validate inputs and test failure paths.	**Comment l'appliquer dans un logiciel d'ingénierie ?** Appliquer cette interface aux pilotes et aux données; isoler les E/S, valider les entrées et tester les erreurs.

Engineering Notes	Notes d'ingénierie
Prefer explicit contracts, deterministic behavior and testable boundaries. Add type hints and documentation when they reduce ambiguity; do not use abstraction only for style.	Privilégier des contrats explicites, un comportement déterministe et des limites testables. Ajouter des indications de type et de la documentation lorsqu'elles réduisent l'ambiguïté; éviter l'abstraction purement stylistique.

Shared engineering example — code remains English only

```
def on_alarm(code):
    print(f"Alarm: {code}")
    shutdown_output()  # supplied by equipment driver
# Register on_alarm with the actual driver API.
```



© AHDS. Portfolio / educational material. Redistribution or reproduction without permission is not authorized.



Section E — Recursion & Functional Programming / Récursivité et programmation fonctionnelle

Chapter 92 — Recursion

Récursivité

Engineering focus / Orientation ingénierie: base cases, recursive progress and Python stack limits. / cas de base, progression récursive et limites de pile Python.

ENGLISH	FRANÇAIS
What is the core purpose of recursion? Recursion solves a problem through smaller instances with a terminating condition.	**Quel est le rôle principal de « Récursivité » ?** La récursivité résout un problème par des appels plus petits avec arrêt.

ENGLISH	FRANÇAIS
What design mistake should be avoided? Detect cyclic structures and avoid excessive recursion depth.	**Quelle erreur de conception faut-il éviter ?** Détecter les cycles et limiter la profondeur de récursion.

ENGLISH	FRANÇAIS
How is this applied in engineering software? Apply this interface to instrument drivers and data pipelines; isolate hardware I/O, validate inputs and test failure paths.	**Comment l'appliquer dans un logiciel d'ingénierie ?** Appliquer cette interface aux pilotes et aux données; isoler les E/S, valider les entrées et tester les erreurs.

Engineering Notes	Notes d'ingénierie
Prefer explicit contracts, deterministic behavior and testable boundaries. Add type hints and documentation when they reduce ambiguity; do not use abstraction only for style.	Privilégier des contrats explicites, un comportement déterministe et des limites testables. Ajouter des indications de type et de la documentation lorsqu'elles réduisent l'ambiguïté; éviter l'abstraction purement stylistique.

Shared engineering example — code remains English only

```
def walk(node):
    yield node
    for child in node.children:
        yield from walk(child)
```



© AHDS. Portfolio / educational material. Redistribution or reproduction without permission is not authorized.



Chapter 93 — Functional Programming Fundamentals

Fondamentaux de la programmation fonctionnelle

Engineering focus / Orientation ingénierie: composition, transformations and predictable data flow. / composition, transformations et flux de données prévisible.

ENGLISH	FRANÇAIS
What is the core purpose of functional programming fundamentals? Functional style composes transformations and minimizes externally visible state.	**Quel est le rôle principal de « Fondamentaux de la programmation fonctionnelle » ?** Le style fonctionnel compose des transformations et limite l'état visible.

ENGLISH	FRANÇAIS
What design mistake should be avoided? Keep conversion and filtering pure when feasible; define data units.	**Quelle erreur de conception faut-il éviter ?** Garder conversions et filtres purs; préciser les unités.

ENGLISH	FRANÇAIS
How is this applied in engineering software? Apply this interface to instrument drivers and data pipelines; isolate hardware I/O, validate inputs and test failure paths.	**Comment l'appliquer dans un logiciel d'ingénierie ?** Appliquer cette interface aux pilotes et aux données; isoler les E/S, valider les entrées et tester les erreurs.

Engineering Notes	Notes d'ingénierie
Prefer explicit contracts, deterministic behavior and testable boundaries. Add type hints and documentation when they reduce ambiguity; do not use abstraction only for style.	Privilégier des contrats explicites, un comportement déterministe et des limites testables. Ajouter des indications de type et de la documentation lorsqu'elles réduisent l'ambiguïté; éviter l'abstraction purement stylistique.

Shared engineering example — code remains English only

```
valid = list(filter(is_valid, samples))
scaled = list(map(to_volts, valid))
```



© AHDS. Portfolio / educational material. Redistribution or reproduction without permission is not authorized.



Chapter 94 — Pure Functions, Side Effects & State

Fonctions pures, effets de bord et état

Engineering focus / Orientation ingénierie: separating calculations from I/O for testability. / séparation des calculs et des E/S pour faciliter les tests.

ENGLISH	FRANÇAIS
What is the core purpose of pure functions, side effects & state? Pure functions return outputs based only on inputs without observable side effects.	**Quel est le rôle principal de « Fonctions pures, effets de bord et état » ?** Une fonction pure dépend uniquement de ses entrées sans effet de bord observable.

ENGLISH	FRANÇAIS
What design mistake should be avoided? Do not read hardware or modify globals inside calibration calculations.	**Quelle erreur de conception faut-il éviter ?** Ne pas lire le matériel ni modifier les globales dans les calculs.

ENGLISH	FRANÇAIS
How is this applied in engineering software? Apply this interface to instrument drivers and data pipelines; isolate hardware I/O, validate inputs and test failure paths.	**Comment l'appliquer dans un logiciel d'ingénierie ?** Appliquer cette interface aux pilotes et aux données; isoler les E/S, valider les entrées et tester les erreurs.

Engineering Notes	Notes d'ingénierie
Prefer explicit contracts, deterministic behavior and testable boundaries. Add type hints and documentation when they reduce ambiguity; do not use abstraction only for style.	Privilégier des contrats explicites, un comportement déterministe et des limites testables. Ajouter des indications de type et de la documentation lorsqu'elles réduisent l'ambiguïté; éviter l'abstraction purement stylistique.

Shared engineering example — code remains English only

```
def compensate(raw, gain, offset):
    return raw * gain + offset
# I/O is performed outside the pure calculation
```



© AHDS. Portfolio / educational material. Redistribution or reproduction without permission is not authorized.



Chapter 95 — Custom Iterators

Itérateurs personnalisés

Engineering focus / Orientation ingénierie: the iterator protocol: __iter__() and __next__(). / protocole d'itération : __iter__() et __next__().

ENGLISH	FRANÇAIS
What is the core purpose of custom iterators? An iterator provides __iter__ and __next__, raising StopIteration when exhausted.	**Quel est le rôle principal de « Itérateurs personnalisés » ?** Un itérateur expose __iter__ et __next__, et signale sa fin par StopIteration.

ENGLISH	FRANÇAIS
What design mistake should be avoided? Bound acquisition count; propagate faults and release device resources.	**Quelle erreur de conception faut-il éviter ?** Limiter les lectures; transmettre les défauts et libérer les ressources.

ENGLISH	FRANÇAIS
How is this applied in engineering software? Apply this interface to instrument drivers and data pipelines; isolate hardware I/O, validate inputs and test failure paths.	**Comment l'appliquer dans un logiciel d'ingénierie ?** Appliquer cette interface aux pilotes et aux données; isoler les E/S, valider les entrées et tester les erreurs.

Engineering Notes	Notes d'ingénierie
Prefer explicit contracts, deterministic behavior and testable boundaries. Add type hints and documentation when they reduce ambiguity; do not use abstraction only for style.	Privilégier des contrats explicites, un comportement déterministe et des limites testables. Ajouter des indications de type et de la documentation lorsqu'elles réduisent l'ambiguïté; éviter l'abstraction purement stylistique.

Shared engineering example — code remains English only

```
class Samples:
    def __init__(self, data): self._items = iter(data)
    def __iter__(self): return self
    def __next__(self): return next(self._items)
readings = list(Samples([1.1, 1.2, 1.3]))
```



© AHDS. Portfolio / educational material. Redistribution or reproduction without permission is not authorized.



Chapter 96 — Generator Functions & yield

Fonctions génératrices et yield

Engineering focus / Orientation ingénierie: lazy production of values with suspended execution state. / production paresseuse de valeurs avec état d'exécution suspendu.

ENGLISH	FRANÇAIS
What is the core purpose of generator functions & yield? A generator yields values lazily and resumes execution between yields.	**Quel est le rôle principal de « Fonctions génératrices et yield » ?** Un générateur produit les valeurs à la demande entre deux yield.

ENGLISH	FRANÇAIS
What design mistake should be avoided? Ensure acquisition terminates and device cleanup occurs on failures.	**Quelle erreur de conception faut-il éviter ?** Garantir la fin des acquisitions et le nettoyage en cas d'erreur.

ENGLISH	FRANÇAIS
How is this applied in engineering software? Apply this interface to instrument drivers and data pipelines; isolate hardware I/O, validate inputs and test failure paths.	**Comment l'appliquer dans un logiciel d'ingénierie ?** Appliquer cette interface aux pilotes et aux données; isoler les E/S, valider les entrées et tester les erreurs.

Engineering Notes	Notes d'ingénierie
Prefer explicit contracts, deterministic behavior and testable boundaries. Add type hints and documentation when they reduce ambiguity; do not use abstraction only for style.	Privilégier des contrats explicites, un comportement déterministe et des limites testables. Ajouter des indications de type et de la documentation lorsqu'elles réduisent l'ambiguïté; éviter l'abstraction purement stylistique.

Shared engineering example — code remains English only

```
def samples(count):
    for _ in range(count):
        yield read_adc()
```



© AHDS. Portfolio / educational material. Redistribution or reproduction without permission is not authorized.



Chapter 97 — yield from & Generator Delegation

yield from et délégation de générateur

Engineering focus / Orientation ingénierie: delegating iteration to another iterable or generator. / délégation de l'itération à un autre itérable ou générateur.

ENGLISH	FRANÇAIS
What is the core purpose of yield from & generator delegation? yield from delegates iteration to a sub-iterator or sub-generator.	**Quel est le rôle principal de « yield from et délégation de générateur » ?** yield from délègue l'itération à un sous-générateur.

ENGLISH	FRANÇAIS
What design mistake should be avoided? Record channel identity and handle failure of each delegated stream.	**Quelle erreur de conception faut-il éviter ?** Identifier les canaux et gérer les erreurs de chaque flux.

ENGLISH	FRANÇAIS
How is this applied in engineering software? Apply this interface to instrument drivers and data pipelines; isolate hardware I/O, validate inputs and test failure paths.	**Comment l'appliquer dans un logiciel d'ingénierie ?** Appliquer cette interface aux pilotes et aux données; isoler les E/S, valider les entrées et tester les erreurs.

Engineering Notes	Notes d'ingénierie
Prefer explicit contracts, deterministic behavior and testable boundaries. Add type hints and documentation when they reduce ambiguity; do not use abstraction only for style.	Privilégier des contrats explicites, un comportement déterministe et des limites testables. Ajouter des indications de type et de la documentation lorsqu'elles réduisent l'ambiguïté; éviter l'abstraction purement stylistique.

Shared engineering example — code remains English only

```
def all_channels():
    yield from samples_ch1()
    yield from samples_ch2()
```



© AHDS. Portfolio / educational material. Redistribution or reproduction without permission is not authorized.



Chapter 98 — Function Decorators

Décorateurs de fonction

Engineering focus / Orientation ingénierie: wrapping callables for logging, timing, validation or policy. / encapsulation d'objets appelables pour journalisation, mesure du temps, validation ou politique.

ENGLISH	FRANÇAIS
What is the core purpose of function decorators?	**Quel est le rôle principal de « Décorateurs de fonction » ?**
A decorator wraps a callable to add behavior without editing its body.	Un décorateur enveloppe une fonction pour ajouter un comportement.

ENGLISH	FRANÇAIS
What design mistake should be avoided?	**Quelle erreur de conception faut-il éviter ?**
Preserve metadata with functools.wraps; never swallow errors silently.	Préserver les métadonnées avec wraps; ne pas masquer les erreurs.

ENGLISH	FRANÇAIS
How is this applied in engineering software?	**Comment l'appliquer dans un logiciel d'ingénierie ?**
Apply this interface to instrument drivers and data pipelines; isolate hardware I/O, validate inputs and test failure paths.	Appliquer cette interface aux pilotes et aux données; isoler les E/S, valider les entrées et tester les erreurs.

Engineering Notes	Notes d'ingénierie
Prefer explicit contracts, deterministic behavior and testable boundaries. Add type hints and documentation when they reduce ambiguity; do not use abstraction only for style.	Privilégier des contrats explicites, un comportement déterministe et des limites testables. Ajouter des indications de type et de la documentation lorsqu'elles réduisent l'ambiguïté; éviter l'abstraction purement stylistique.

Shared engineering example — code remains English only

```
from functools import wraps
def traced(fn):
    @wraps(fn)
    def wrapper(*args, **kwargs):
        print(f"Calling {fn.__name__}")
        return fn(*args, **kwargs)
    return wrapper
@traced
def self_test(): return True
```



© AHDS. Portfolio / educational material. Redistribution or reproduction without permission is not authorized.



Chapter 99 — Function Introspection, Documentation & Robust Function Design

Introspection, documentation et conception robuste des fonctions

Engineering focus / Orientation ingénierie: __name__, __doc__, docstrings, signatures, annotations and basic callable introspection. / __name__, __doc__, docstrings, signatures, annotations et introspection de base des objets appelables.

ENGLISH	FRANÇAIS
What is the core purpose of function introspection, documentation & robust function design? Introspection exposes function signatures, annotations and documentation at runtime.	**Quel est le rôle principal de « Introspection, documentation et conception robuste des fonctions » ?** L'introspection expose signatures, annotations et documentation à l'exécution.

ENGLISH	FRANÇAIS
What design mistake should be avoided? Inspect public contracts but do not rely on undocumented implementation details.	**Quelle erreur de conception faut-il éviter ?** Examiner les contrats publics sans dépendre de détails internes.

ENGLISH	FRANÇAIS
How is this applied in engineering software? Apply this interface to instrument drivers and data pipelines; isolate hardware I/O, validate inputs and test failure paths.	**Comment l'appliquer dans un logiciel d'ingénierie ?** Appliquer cette interface aux pilotes et aux données; isoler les E/S, valider les entrées et tester les erreurs.

Engineering Notes	Notes d'ingénierie
Prefer explicit contracts, deterministic behavior and testable boundaries. Add type hints and documentation when they reduce ambiguity; do not use abstraction only for style.	Privilégier des contrats explicites, un comportement déterministe et des limites testables. Ajouter des indications de type et de la documentation lorsqu'elles réduisent l'ambiguïté; éviter l'abstraction purement stylistique.

Shared engineering example — code remains English only

```
import inspect
def measure(channel: int) -> float:
    """Return simulated voltage for a channel."""
    return float(channel)
print(inspect.signature(measure), measure.__doc__)
```



© AHDS. Portfolio / educational material. Redistribution or reproduction without permission is not authorized.